FRAMEWORK

面向跨环境异构语义保持的目标感知反混淆折叠保护分析框架 (R97 Framework Overview)

针对离线求值导致环境敏感分支误剪与运行时崩溃问题 · 五层科研闭环模型：输入 ➔ 特征解耦 ➔ 守门模型 ➔ 验证闭环 ➔ 保语义输出

第一层: 输入层

Input Layer

1. 异构受保护/混淆源码输入 (Protected JavaScript Code):

- 移动小程序 `if(typeof wx) wx.req()`
- 前端 Web WPT `window.localSt...`
- 后端 Node `process.env`

2. 外部客观部署证据输入 (External Manifest Proofs):

- 小程序配置清单 `project.config.json`
- Web 清单 / 指纹 `index.html, manifest`

第二层: 特征分析层

Feature Extraction

代码侧：AST 作用域与依赖解析 (AST Scope & Alias Engine)

- 词法作用域链遍历：排除局部变量与函数形参遮蔽伪依赖
- 常量别名递归追踪：追溯 client ➔ wx 动态引用链

➔ 必需能力状态 (required): { definite (确信依赖), possible, not_required }

🔒 正交隔离防火墙 (Required ⊥ Target)

科学不变量：绝对禁止循环推断！

- 代码内部写了 wx，仅代表依赖需求 (required)；
- 严禁借此反推目标环境！目标必须由外部独立给出。

证据侧：外部目标环境解析 (Target Runtime Re

- 外部事实核验：解析 project.config.json 与环境指纹
- 排除多源配置冲突，防范代码自身特征反哺

➔ 目标确认状态 (target): { confirmed (强证实), co

★ 核心模型层 ★ Proposed Model

Tri-State Guard

1. 语义契约模型 C (Contract Builder)

- 用法上下文驱动投影：按运算符提取最小保真维度

typeof 维度 | truthiness 维度 | return_val 维度

非确定性语言内建陷阱拦截:

命中 Date.now() / Math.random() 等时变调用
➔ 无法自等 ➔ 强制要求 return_val ➔ 守住未决 UNKNOWN

- 比较谓词绑定：predicate = strict_equal
- 作用域有效性范围签名绑定 (SHA-256)

➔ 输出契约: C = ⟨strict_equal, requiredDims⟩

2. 4 宿主差分探针实测矩阵 (Empirical Probing)

① 语言基线 VM	② 原生 Node.js	③ Edge 浏览器	④ 微信 DevTools
Node vm 纯净环境	v20+ (553条差分)	WPT (780条差分)	SDK 3.17 (785差分)

```
[实测冲突] typeof wx:
VM/Node: 'undefined' ≠ 微信真机: 'object'
➔ 捕获真实异构差分 ➔ 硬触发 PROTECT
```

```
[实测等价] Math.abs(-5) === 5:
4 宿主读数皆为 true (strict_equal)
➔ 严格等价且证据可折叠 ➔ 放行 FOLD
```

- 差分证据库 E: 1248 实体快照 / SHA-256 指纹时效绑定
- 非对称外推原则: 存在可外推，不存在绝对不可外推 (dropAbsence)

➔ 输出证据向量: evidenceRecords 供第 7 步比对

3. 八步安全判定核心 D(req, target, C, E)

① 绑定检查 ➔ ② 契约完备性 ➔ ③ 必需明确性 ➔ ④ 目

⑤ 宿主缺失核验 ➔ ⑥ 探针证据时效 ➔ ⑦ 严格投影比对

FOLD — 等价安全放行 (785 差分点 0 误放行，折
所有确认目标 strict_equal 且 foldEligible ➔ 折

PROTECT — 冲突硬拦截 (实测挽救 762 处真实环
已确认目标检测到物理差分 ➔ 强制锁死分

UNKNOWN — 保守安全未决 (Fail-Safe 疑点利益归
输出细粒度归因: binding (92%) / runtime

第四层: 验证闭环层

Double-Oracle

1. Level-2 真机动态执行观测等价性验证 (OBsmith §3.4 规范):

- 语料实际放行 1,338 程序点，真机动态执行验证 386 点；
- 提取规范化异常标识符 (ReferenceError:target=wx)，比对终结状态与值投影；

✸ 终极闭环指标：变换前后真机执行行为不一致率：0.00%！

2. 多元三域基准评测实证指标 (Benchmark Metrics):

- 全文件 4 宿主生产口径 (3,652 样本): FOLD 32.53% | PROTECT 20.87% | 可判定率 53.40%
- 全文件 3 宿主基准 (Node+Edge): FOLD 放行率 43.32% | 可判定率 47.10%

✸ 核心安全指标：全量 785 条跨宿主实测差分中，实测不安全折叠 (unsafeFold) 严格为

第五层: 输出与守护层

Output & Guard

传统反混淆器 (无 R97)

- 离线 Node 单沙箱直接求值：typeof wx ➔ 恒假
- 误触发死代码剪枝：**将环境适配分支完全删除！**

☣ 部署真机后果：核心能力丢失，抛出 TypeError / ReferenceError 运行时崩溃！

集成 R97 守门保护

- 实测探针证实真机存在 wx ➔ 输出硬判定 PROTECT
- **强制拦截折叠！原汁原味保留分支结构；纯计算安全放行 FOLD**

☑ 真机部署成果：微信/前端/后端 100% 正常运行，功能零缺失，保语义安全反混淆输出！